

Perfective allomorphy and negative neutralisation in Bambara: a nanosyntactic approach

Karen De Clercq

CNRS/Laboratoire de Linguistique Formelle/ Université Paris Cité
karen.de-clercq@cnrs.fr

Abstract

Bambara (Mande) displays perfective (henceforth PFV) allomorphy conditioned by transitivity properties of the verbal predicate. The allomorphy is neutralised under negation (NEG). I present an analysis of these data in Nanosyntax (Starke 2009, Caha 2009), which relies on the size of lexically stored items (Starke 2014). The PFV allomorphy is accounted for by proposing that the lexical structure of the intransitive PFV marker *-ra* has its bottom feature in VOICE, while the transitive PFV *ye* has a lexical structure with a bottom feature that sits higher in the structure, i.e. in ASP. Negative neutralisation follows naturally from the proposal, if one adopts a low position in syntax for NEG, i.e. just above Voice. The paper also shows how the Nanosyntactic algorithm can help to diagnose the syntactic position of negation in a highly isolating language like Bambara.¹

1 Introducing the data

Bambara (also called Bamana or Bamanankan) is a language spoken in West Africa that belongs to the Eastern Manding varieties, which are a dialect cluster that is part of the Central subbranch of the Western Mande language family (Idiátov 2012). I am focusing on Standard Bambara in this paper relying for the data on Dumestre (2003), Koopman (1992), Ermisch (2013), Creissels (2007) and data collected via my informant Dieydi Sylla.

Bambara has two ways to mark perfective or completive aspect: (i) the suffixal marker *-ra* (which becomes *-la* after /r/ or /l/ and *-na* after /m, n/ or a nasal vowel; Hewson 2016: 53), and (ii) the free morpheme *ye* (Dumestre 2003). Unergative (1a) and unaccusative (2a) verbs take *-ra*, and are incompatible with *ye*, (1b)-(2b). Transitive verbs with NP complements take the marker *ye* (3), and are incompatible with *-ra*, (3b).²

¹This work has been carried out within the framework of the ANR-22CE28-0006 SNAG project supported by the French National Research Agency. I want to thank the audiences of MaS2 (UCLA, 2022), GLOW47, the Syntax Colloquium at Goethe Universität Frankfurt-am-Main (May 2025) and the Asya Fest (June 2025) for their useful comments and questions. Special thanks to Guido Vanden Wyngaerd, Hilda Koopman, and Michal Starke for their feedback on some parts or versions of this paper. All errors are of course mine. All feedback is very welcome.

²Bambara has lexical and grammatical tone. However, since the tonal differences do not bear on the issues discussed in this paper, I leave them undiscussed and unmarked in the examples. I refer the reader to Creissels (1988, 1992), Dumestre (2003) for more information on the tonal system.

- (1) a. *A kasi- ra.*
 s/he cry PFV
 'S/he cried.'
 b. **A ye kasi.*
 s/he PFV cry

- (2) a. *A taa- ra.*
 s/he go PFV
 b. **A ye taa.*
 s/he PFV go

(Koopman 1992: 559-560)

- (3) a. *Birama ye liburu kalan.*
 Birama PFV book read
 'Birama has read the book.'
 b. **Birama kalan- ra liburu.*
 Birama read PFV book

Under negation this sensitivity to transitivity gets neutralised, as illustrated in (4): both the unaccusative verb in (4a) and the transitive verb in (4b) take the negative marker *ma* (Ermisch 2013). Actually, *ma* is not just a negator, it is a fusional marker expressing at least perfectivity and negation.

- (4) a. *Fali ma sa.*
 donkey PFV.NEG die
 'The donkey has not died.'
 b. *Birama ma liburu kalan.*
 Birama PFV.NEG book read
 'Birama has not read the book.'

It is important to mention that there is another fusional negator used in the imperfective (IPFV), i.e. *tɛ*, whose positive counterpart is the partially syncretic *bɛ* (De Clercq 2020b).

- (5) a. *Sanu bɛ baara kE*
 Sanu PRS.IPFV work do
 'Sanu works.'
 b. *Sanu tɛ baara kE*
 Sanu PRS.IPFV.NEG work do
 'Sanu does not work.' (Ermisch 2013: 2)

Unlike what we see in the PFV, there is no split related to transitivity in the IPFV: the same form *bɛ* is used with transitive and intransitive verbs alike, as in (6).

- (6) a. *Séku bɛ tiga sɛnɛ*
 Séku PRS.IPFV groundnut cultivate
 'Sékou cultivates groundnuts.' (transitive)
 b. *Séku bɛ boli*
 Sékou PRS.IPFV run

‘Sékou runs.’ (intransitive) (Creissels 2007: 2-5)

I argue in this paper that these markers are more than just aspectual markers. They are Tense-Aspect-Mood-Polarity (henceforth TAMP)-portmanteaus, i.e. fusional morphemes that lexicalise TAM-related properties, including polarity. Confirmation for this comes from their inability to be stacked within the same clause, as (non-exhaustively) illustrated in (7).

- (7)
- a. **Sanu tɛ be baara kE*
Sanu IPFV.NEG IPFV work do
 - b. **Sanu be tɛ baara kE*
Sanu IPFV IPFV.NEG work do
 - c. **Birama ma bɛ liburu kalan.*
Birama PFV.NEG IPFV book read
 - d. **Birama ma ye liburu kalan.*
Birama PFV.NEG PFV book read
 - e. **Birama ye ma liburu kalan.*
Birama PFV PFV.NEG book read

The data can be summarized as in (8).³ What is immediately striking in (8) is that 1) there are only two different markers for the PFV, and 2) there is only one suffixal marker in the entire paradigm, i.e. the PFV *-ra*. All other TAMP markers precede the VP.

(8)

	PFV	NEG + PFV	IPFV	NEG + IPFV
INTRANS	-ra	ma	bɛ	tɛ
TRANS	ye	ma	bɛ	tɛ

Before we move on, it is important to mention that what we refer to as the perfective (PFV) can involve two different readings: one that overlaps with the simple past in English, and one that overlaps with the present perfect tense. Dumestre (2003: 210) says the following about the perfective/completive marker in Bambara: “The perfective describes an event as anterior to the reference point, with or without consequence for the latter.”⁴ I follow this idea in this paper, and in what follows I will use the translation of my sources whenever I present the data. I will also keep using the label perfective to refer to the relevant marker. The interpretational ambiguity will not be morphosyntactically reflected in the analysis presented in section 3-4. The assumption is that the ambiguity results from how the event and reference times, encoded in the TAM morphology, are related to the speech time, without taking a stand on whether the latter is morphosyntactically or contextually represented across languages (Klein 1992, Giorgi & Pianesi 1997). If the event and reference time precede the speech time, a simple past reading arises. If the event time precedes the reference time while the latter coincides with the speech time, a Present Perfect reading arises.⁵ This

³We abstract away in this paper from the past (*tun*) and future (*(-)na*) tense markers, which can combine with these TAMP-markers to give rise to past imperfective, past perfective, and future readings (see Dumestre 2003, De Clercq 2020b). We also do not discuss the fact that *tɛ* and *bɛ* can be used in different types of copular constructions, where they are syncretic with the imperfective marker. We hope to come back to this issue in future work, but for now, we refer the reader to Dumestre (2003: ch3) for discussion.

⁴Original text: L’accompli décrit un procès envisagé comme antérieur au moment de référence, avec ou sans conséquences pour celui-ci.”

⁵A fine-grained crosslinguistic analysis of syncretism patterns between the simple past and the present

paper’s focus is on providing 1) an account of the allomorphy conditioned by transitivity in the perfective, and 2) the neutralisation of that allomorphy under negation. We will show how the allomorphy follows from the size of stored lexical trees, and how the neutralisation and concomitant syncretism in NEG.PFV can be captured thanks to position of negation in the FSEQ and the Nanosyntactic algorithm.

In order to achieve this goal, we will first have a look at Koopman (1992)’s account for the perfective allomorphy in Bambara in section 2. We will adopt the core of her analysis, i.e. that the allomorphy in the perfective is linked to the selection of an argument with accusative case, but we will also show that her account does not explain the neutralisation facts under negation. We will take this as an argument to propose a new analysis, couched in Nanosyntax (Starke 2009), which we introduce in section 3-4, alongside an analysis of the data. We analyse the split in transitivity as Voice-conditioned allomorphy that arises as a consequence of the lexical size and shape of the PVF markers in Bambara and the interactions with the lexicalisation algorithm. The neutralisation will be analysed as a consequence of the structure of the negative perfective portmanteau *ma* on the one hand, and the position of NEG in the structure on the other hand. In Section 5 we briefly elaborate on how the selection of internal and external arguments could be fit into the proposed analysis. Section 6 wraps up and concludes.

2 The background

Koopman (1992: 564) discusses the split in the distribution of the perfective marking and summarizes it as follows:

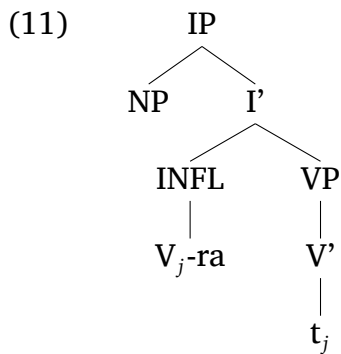
- (9) a. *ye* occurs if V assigns structural accusative case.
- b. *-ra* occurs if V does not assign structural accusative. (Koopman 1992: 562, (8))

She explains the data in (1)-(2) as a consequence of the Case Filter. More specifically, she argues that Case chains do not exist in Bambara, i.e. verbal traces cannot assign case and movement of a transitive verb would leave the object unlicensed.

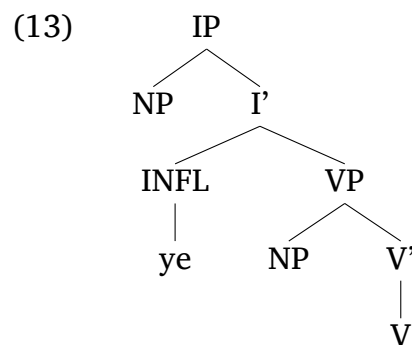
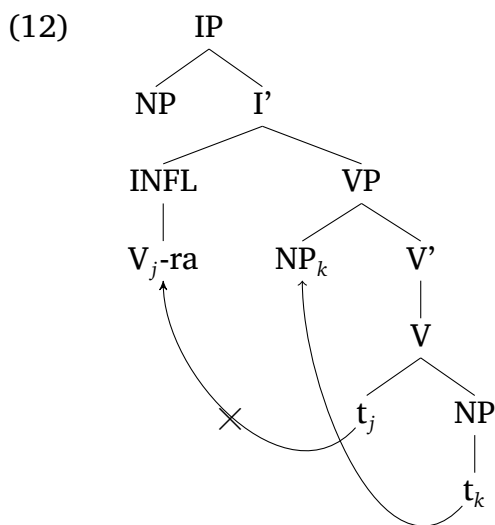
- (10) “If the verb moves, the direct object NP would not be assigned Case, and a Case filter violation would arise. If the moved verb cannot assign Case from INFL - a reasonable assumption given the proposal that Case is assigned strictly leftward - the direct object can only be assigned accusative Case if the verb remains in situ, and another INFL *ye*, which I consider to be a suppletive form for *-ra*, appears.” (Koopman (1992: 564))

The idea is that when the verb has no internal argument to assign case to, it can move to INFL and pick up *-ra*. The tree structure in (11) from Koopman (1992: 563) illustrates this.

perfect could lead to a morphosyntactic implementation, but we refrain from that for the sake of this paper.



However, the transitive verb needs to stay in situ to assign structural accusative case to the object in a SpecHead configuration. The structure from Koopman (1992) in (12) illustrates that movement is not allowed, because that would leave the object unlicensed.



Since the verb cannot move to INFL, *ye*, which Koopman considers a suppletive form of *-ra* (see quotation in (10)), appears in INFL ((13)), in a similar way as *do* is inserted in English, (13). *-ra* cannot appear in this case, since *-ra* is in need of a verbal host.

Koopman provides a lot of data to support her hypothesis that the allomorphy is linked to accusative case assignment. A first set of data comes from transitive verbs with PP-complements. These verbs also take *-ra*, not *ye*, as in (14a)- (14b)(from Koopman 1992). The idea is that in this case the preposition assigns case to the object, and hence the verb is free to move.

- (14) a. *N son- na a ma.*
 I agree PFV it to
 I agreed to it, I accepted it.
 b. **N ye son a ma.*
 I PFV accept it to

A second set of data comes from passive verbs: they take *-ra*, (15), and pattern with intransitives in not being compatible with *ye*. This is expected under Burzio's generalisation, i.e. passives do not assign structural accusative case, because there is no external θ -role assigned

to a subject (Burzio 1986, Chomsky 1981), i.e. the subject gets suppressed leading to the promotion of the internal θ -role and it receiving nominative case.

- (15) *Ji min- na sisan (den fε).*
 water drink PFV now child by
 'The water has been drunk now (by the child).' (Koopman 1992: 560)

A third set of data comes from verbs that combine with the causative marker *la-*: in that case the perfective marker is *ye* again, regardless of the argument structural properties of the verb without *la-*.⁶ The reason for this in Koopman's proposal (Koopman 1992: 567) is that the causative assigns an external theta-role and takes a VP complement, and hence assigns case to the NP it governs, forcing *ye* to appear again in INFL. The data in (16) illustrate a causatively marked transitive verb with a PP-complement, (16a), a causatively marked unergative verb, (16b), and a causatively marked unaccusative verb, (16c) (from Koopman 1992: 566).

- (16) a. *Bala ye den la-bó i ye.*
 Bela PFV child CAUS-visit you at
 'Bala made the child visit you.' [caus + V + PP]
 b. *A ye den la-kasi*
 she PFV child CAUS-cry
 'She made the child cry.' [caus + unerg V]
 c. *A ye den la-taa (sugu la).*
 s/he PFV child CAUS-leave market to
 'She made the child go to the market.' [caus + unacc V]

A fourth set of data comes from passivized causatives: in that case *-ra* kicks in, (17). The fact that the causative can be passivized follows from the fact that it can assign an external theta role, and Case. If the internal argument gets promoted to subject position, and no case assignment happens, as is the case in passives, the entire verbal complex can move to INFL to pick up *-ra*.

- (17) *Den lakasi-ra sisan (muso fε).*
 child make-cry-PFV now woman by
 The child has been made to cry now (by the woman).

An issue that remains undiscussed in Koopman's account however is why the transitive vs intransitive distinction disappears under negation in the perfective, i.e. how is it possible that the presence of negation in the structure neutralises the transitive/intransitive split that we observed for affirmative perfective sentences? Put differently, why is (18a) ungrammatical, and why is (18b), repeated here from (4a), grammatical?

- (18) a. **Fali sa -ma.*
 donkey die PFV.NEG
 'The donkey has not died.'

⁶Note that the case-marked object of complex causative verbs must be the theme of the thematic verb and cannot correspond to its agent. I refer the reader to Koopman (1992) for further discussion of this property.

- b. *Fali ma sa.*
 donkey PFV.NEG die
 ‘The donkey has not died.’

If the absence of case chains is really what is at the heart of this allomorphy pattern that we see in the perfective, then it remains unexplained why under negation the verb always seems to stay in the VP, i.e. there is always a TAM-marker filling the INFL slot, regardless of whether the verb needs to assign accusative case or not.

While we do want to keep the essence of Koopman’s account, i.e. that case plays an important role in the allomorphy that we see in PFV, something more needs to be said to be able to account for the neutralisation of the transitivity split in NEG.PFV. Since the relevant markers under discussion are fusional markers, we opt for Nanosyntax to analyse the data. Nanosyntax has shown itself to be well-equipped to capture portmanteaus, and allomorphy in terms of the size and shape of lexically stored morphemes (Caha et al. 2019). In section 3, we introduce the basics of Nanosyntax, the functional sequence (fseq), and the lexical structures of the various TAMP morphemes under discussion. In section 4 we discuss the detailed derivation of the affirmative PFV for transitives and intransitive verbs, followed by the neutralisation under negation.

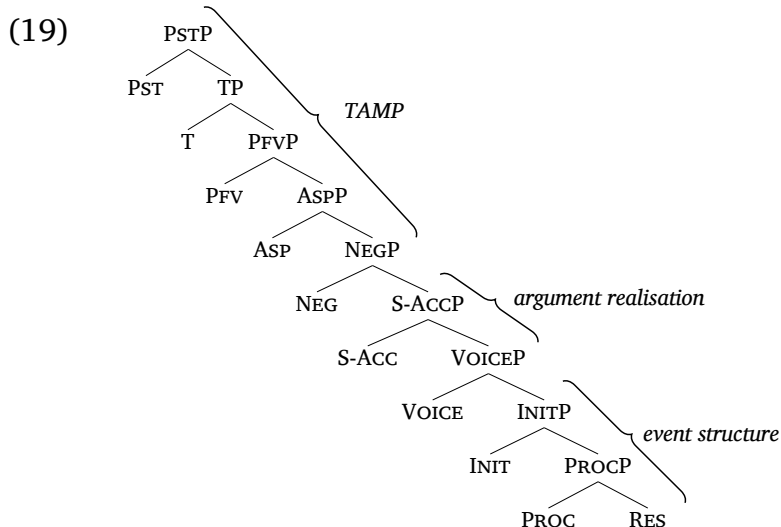
3 Analysis: The structure of the FSEQ and lexical items

There are two essential components to understanding the split that we see in the positive PFV and its neutralisation under negation. The first component is the structure of the FSEQ and the internal structure of the lexical items for TAMP in Bambara. The second component is how the universal functional sequence gets mapped onto these language particular lexical items by means of the lexicalisation algorithm. In this section we discuss the first component, i.e. the universal FSEQ and the language specific lexical items. In the next section, when we discuss the detailed derivations, we will show how the Nanosyntactic algorithm mediates between these two.

We adopt a functional sequence as in (19). I take the layers Result (RES), Proces (PROC) and Initiator (INIT), from Ramchand (2008), to be responsible for the event structural properties of the verbal predicate. The INIT head introduces the causation event. The PROC head specifies the nature of the change or process, whereas RES gives the result of the event. In addition, and unlike Ramchand (2008), I propose that there are at least two layers in the verbal domain that enable the predicate to combine with its arguments, i.e. a layer VOICE in case the predicate combines with one argument, and a layer Select Accusative (S-ACC) (Caha 2009), which stands for “select an accusatively marked argument”. Note that these layers are part of the morphosyntax of the verbal domain: each of these layers will eventually instruct the syntax to select a fitting argument. The arguments themselves will be in separate workspaces. We will briefly reflect on some of the options concerning where the arguments could be argued to be introduced in section 5.

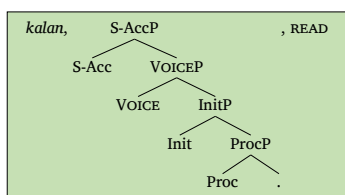
NegP in (19), i.e. Sentence Negation, has a low position in Bambara on top of the Voice layers, and below default ASP. We show in section 4.2 how we can deduce - on the basis of the Nanosyntax algorithm and the allomorphy in Bambara - that this low position must indeed be the position for negation. Above default ASP, there is a layer for the perfective, PFV, and then there are tense layers, of which we only represent TP and PSTP here. The

semantics of this type of fine-grained structures arises via a Gricean logic (Starke 2022, De Clercq et al. to appear): if PFV is left out, the interpretation is IPFV, if PST is left out, the interpretation is PRS.

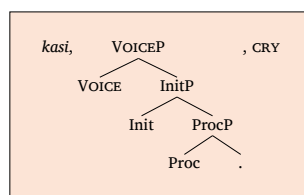


The FSEQ in (19) gives the universal ordering of features that we assume here. However, in the lexicon of Bambara these features are packaged together in chopped chunks and stored with phonological and conceptual information. Lexical items in Nanosyntax are thus a “storage place for memorised links between syntactic structures on the one hand and sound/meaning on the other hand” (Caha 2022: 10). We will work with two Bambara toy verbs in this paper to explain the data: a transitive verb *kalan* ‘read’, (20), and an intransitive verb *kasi* ‘cry’, (21). Both lexical items consist of phonological information, followed by a tree structure, and then conceptual information (in small caps). In most of the paper we will zoom in on the lexical structure. The two verbs that we use to illustrate Bambara verbal predicates in (20) and (21) lexicalise not only the layers of the Ramchandian event structure but also voice related instructional layers. Crucially, we propose that transitive verbs in Bambara can lexicalise Voice and S-ACC, while intransitive verbs lack the layer S-ACC.

(20)



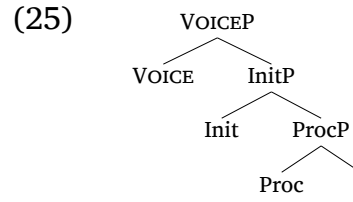
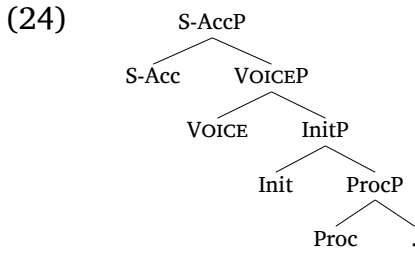
(21)



Thanks to the Condition on Matching (22), from which the Superset Effect (23) follows (Starke 2009, see De Clercq et al. 2025 for discussion of how), a lexical item as in (20) can lexicalise both the syntax of a transitive verbal structure (24), as well as the syntax of an intransitive verbal structure (25). It can also lexicalise any other syntactic structure that is a subconstituent of its lexical structure. We will discuss this in more detail in the next section, when we go over the derivation in detail.

- (22) A lexically stored constituent L matches a syntactic phrase S iff S is identical to L. (De Clercq et al. 2025)

(23) A lexically stored tree L matches a syntactic node S iff L contains the syntactic tree dominated by S as a subtree. (Starke 2009)



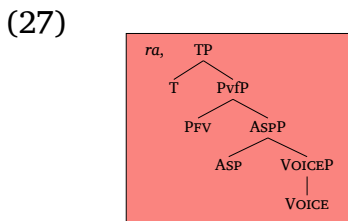
Before we move to the lexical structure of the TAMP markers, we take another look at them in Table (26). Almost all the TAMP-markers are in blue. Those precede the VP. The PFV marker, *-ra*, is a suffix and it is the only marker that follows the verb.

(26)

	PFV	NEG + PFV	IPFV	NEG + IPFV
INTRANS	<i>-ra</i>	<i>ma</i>	<i>be</i>	<i>te</i>
TRANS	<i>ye</i>	<i>ma</i>	<i>be</i>	<i>te</i>

Starke (2018) proposed that the distinction between morphemes that precede, i.e. PRE elements, and morphemes that systematically follow, i.e. POST elements, is lexically encoded. PRE elements are conceived of as independent workspaces, very much like free morphemes, that have a binary bottom. As the reader may have noticed, the bottom of the lexical trees in (20) and (21) consists of a dot, and PROC, i.e. it is also a binary bottom, because these roots are also independent workspace.⁷ POST elements on the other hand have a unary bottom, and - as we will see when we go over the derivations - they are the consequence of evacuation movement (see Section 4). One of the crucial TAMP-markers *-ra* is thus a POST marker, while all the other markers are PRE-makers.

The lexical item for *-ra* is in (27) and it has a unary foot, to capture its suffixal (POST) behaviour. It consists of features that reach into the argument realisation layers, more in particular its bottom feature is VOICE. It is the fact that *-ra* is specified for VOICE, but crucially not for S-ACC that will be key in deriving the transitivity split and allomorphy. The presence of VOICE in the lexical structure of (27) captures the fact that *-ra* only occurs with intransitive verbs, i.e. verbs lacking the S-ACC feature between VOICE and ASP.



On the basis of the fact that the markers in blue in (26) all precede the main predicate, and do not follow it, unlike *-ra* we propose that they have a binary foot, just like *kalan* and

⁷Starke (2024) recently proposed that the right-most bottom feature in a new workspace is an empty feature, which is represented by a dot, and which combines with the first feature that will provide the label for the first node. From a mathematical point of view, it would make sense to refer to this dot as the empty set. However, another common linguistic interpretation of the term *set* is that it can contain something, i.e. it is a container. However, the dot never contains anything, and crucially should not contain anything. So we refer to it as an empty feature for now. Thanks to M. Starke for discussing this with me. All errors are mine.

kasi. All these TAMP-markers will thus involve the formation of a new workspace. How this is done exactly will be discussed in more detail in section 4. Due to the fact that the markers in blue are insensitive to the transitive-intransitive split, we deduce that they cannot realise Voice/Case-related features and hence must lexicalise features that sit higher in the FSEQ, (19), i.e. above S-ACC in (19).

The lexical items for *ye* and *ma*, i.e. the PFV markers that precede, are in (28)- (29). The positive PFV *ye*, (28), consists of default ASP, PFV and T, but lacks NEG, while *ma*, (29), has and additional NEG as its bottom feature.



Note that there are two lexical items for the positive PFV, one which has no VOICE features in its lexical structure, i.e. (28), and one which has, (27). This difference in lexical structure will - together with the algorithm that we will discuss in section 4, account for the allomorphy facts in the PFV. The sole negative PFV lexical item has no VOICE-related features in its internal structure, (29), which points to why there is a neutralisation of the transitivity split under negation. For completeness sake, and because these elements will be relevant in the derivation of the PFV, we also provide the lexical items for the remaining blue markers in Table (26), i.e. the positive IPFV, (30), and NEG.IPFV, (31). Just like with the NEG.PFV *ma* there is only one marker for the IPFV and one marker for the NEG.IPFV. This means that these markers are not sensitive to the argument structural properties of the predicate. Therefore, we again adopt a lexical structure for these markers that has a bottom feature that sits higher than S-ACC in the FSEQ. The structure for *bε* is in (30) and for *tε* in (31).



A summary of all the relevant lexical items is provided in Table 1. The table is a lexicalisation table which informally shows how the different lexical items can be matched against the functional sequence. The different features that make up the functional sequence is the top row, with the highest feature in the tree structure on the left. The different rows depict which parts of the FSEQ are lexicalised by which morphemes. Black cells indicate that particular layers are absent for the structure under consideration in that row. Note that this lexicalisation table does not depict the full complexity of the derivation: for instance, the row of the INTRANS.PFV only provides us with the morpheme for *-ra*, but does not explain why this morpheme cannot combine with a transitive verb, nor how it exactly combines with *kasi*.

	T	PFV	ASP	NEG	S-ACC	VOICE	INIT	PROC
TRANS V					kalan	'read'		
INTRANS V						kasi	'cry'	
TRANS.PFV	ye							
INTRANS.PFV	ra							
NEG.PFV	ma							
IPFV	bɛ							
NEG.IPFV	tɛ							

Table 1: Overview of lexical items

The detailed step-by-step analysis that we will provide in section 4 will illustrate the details of how the internal structure of the verbs and the TAMP markers will contribute to deriving the split and neutralisation that we see in the PFV. Moreover, it will be shown how the Nanosyntactic algorithm helps to map the universal syntax (the FSEQ) to the language particular lexical items of Bambara.

4 Analysis: the derivations

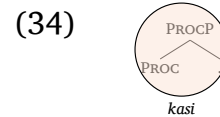
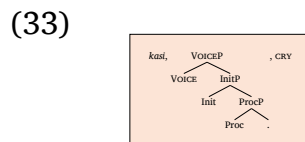
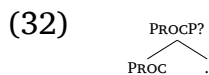
4.1 The derivation of PFV

In this section we will first show how we can derive the allomorphy in the positive PFV, 4.1, and then we will move on to account for how the negative neutralisation arises in section 4.2.

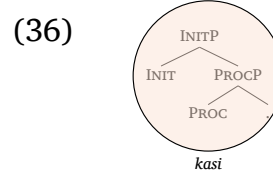
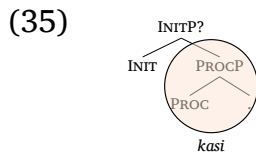
4.1.1 The derivation of INTRANS.PFV *kasi-ra*

We will start with the derivation of an intransitive predicate. The syntax starts merging the relevant features. For a predicate like *kasi* 'cry' the empty feature will be combined with the first feature of an activity denoting predicate PROC, as in (32). The reader may protest that the FSEQ in (19) started with RES, but layers can be skipped if the action that needs to be talked about does not yield a result state. In this particular case, we want to talk about crying, which does not inherently yield a result state, and hence there is no RES in the structure.

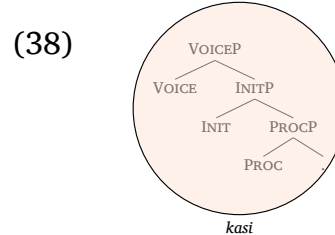
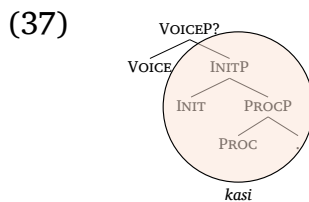
Lexicalisation in Nanosyntax is extremely cyclic and phrasal (see Starke 2009, Caha 2009, Baunaz & Lander 2018 for discussion): this means that after every step of merge, at every singly phrasal level the syntax will interface with the lexicon to find a matching lexical item that can lexicalise the structure. At the level of PROCP in (32) the syntax will interface with the lexicon to find a lexicalisation. There are several lexical items that can lexicalise this structure, but since we want to talk about *cry*, the lexical item in (21), repeated here in (33) will be picked, and the syntax in (32) will be lexicalized. We illustrate successful lexicalisation by means of circled node, as in (34).



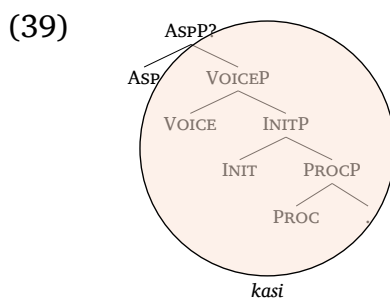
The next feature in line is an Initiator feature (Ramchand 2008), INIT. Just like before, the syntax will interface with the lexicon at the level of INITP, (35). (33) is still the right candidate, since its lexical structure is a Superset of the syntax in (35), in conformity with the Condition on Matching and its concomitant Superset Effect, (23). In this case, (33) can override the previous lexicalisation, leading again to a successful lexicalisation, depicted in (36).



Note that override by means of a lexical item with a different concept would not be allowed at this point anymore. Once a particular root with a particular concept is chosen, one stays faithful to that root and concept (see also De Clercq et al. (to appear), Caha et al. (2019) for more discussion of this issue.) Since there is a match, syntax can continue merging structure. The next feature up in the FSEQ, (19), is Voice. Just like before at the level of the phrase, i.e. VoiceP, the syntax, (37), will interface with the lexicon, and (33), is a perfect match, leading to successful lexicalisation, (38).



Syntax keeps merging. Since we are doing an intransitive structure, S-ACC will not be part of the numeration, so the next feature in line that will be merged is default ASPECT (see (19)). ASP is merged, and again at the level of ASPP the syntax interfaces with the lexicon, (39). However, at this point there is no lexicalisation to be found in Table 2, which repeats our mini-lexicon of Bambara. There is no lexical item available that can lexicalise this entire chunk in (39).



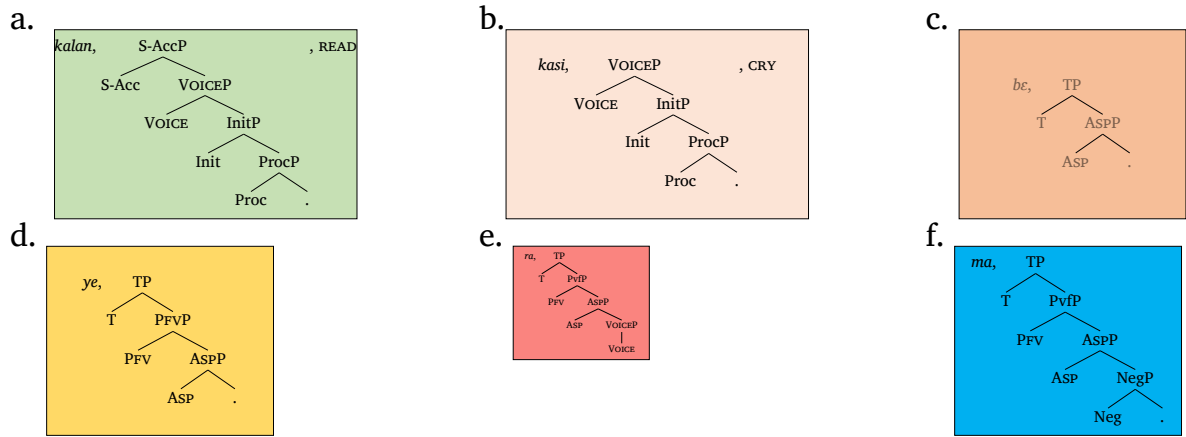


Table 2: Bambara mini-lexicon

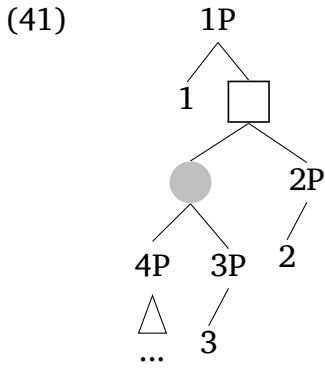
This is where the Nanosyntactic algorithm (40) comes in (see Starke 2018, De Clercq et al. 2025 for discussion). The algorithm is an ordered set of manipulations that the syntax can perform on the structure in order to lexicalise a newly merged feature. In this case, the syntax will need to do a manipulation to the structure in order to be able to lexicalise ASP.

The first step in (40) is Merge, followed by an attempt at lexicalisation of the newly merged feature. This step of the algorithm was successful until (37). However, this does not work for (39), hence the next step, i.e. an evacuation movement, as in (40b), needs to take place.

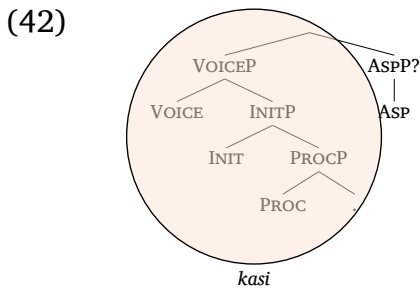
(40) Merge-F

- a. Try to lexicalize
- b. If fail, evacuate the closest labeled node that contains the tree foot (and repeat with successively higher nodes if need be)
- c. If fail, move a complex left branch (CLB) that can provide the feature.
- d. If fail, backtrack.
- e. If fail, spawn a new workspace (CLB) providing F and merge that with the current derivation, projecting F to the top node.
- f. If fail, backtrack (modified after Starke 2018, De Clercq 2020a, De Clercq et al. 2025)

Step (40b) can involve several evacuation attempts, in the sense that you first try to evacuate the closest labeled node that contains the tree foot, which is 4P in (41) (tree structure from De Clercq et al. 2025). If that is unsuccessful, you pied-pipe, leading to the evacuation of the circled node. If that is unsuccessful, you try the next higher node, which is the boxed node in (41). In other words, evacuation follows an economic principle to move as little material as possible first, and to only successively enlarge the portion of material that gets evacuated.

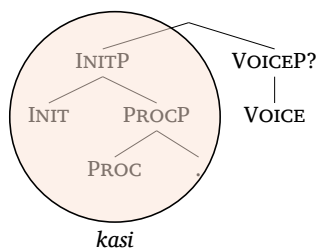


I refer the reader to De Clercq et al. (2025) for more discussion of this issue. For the tree in (39) the node that is closest to the newly merged F and that contains the tree foot is VOICEP (i.e. the complement of ASP). The syntax will thus perform a lexicalisation-driven evacuation movement and the resulting structure is in (42). At this point, ASP sits on the right branch, as new constituent. Syntax interfaces again with the lexicon, to check whether there is a lexicalisation for this feature. However, the lexicon in Table 2 does not contain a lexical item that has ASP as a unary bottom, which is what we need in order to lexicalise the right branch of (42). Nor is there a lexical item that contains the entire structure of (42).

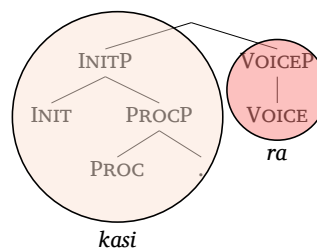


I want to remind the reader here that we want to derive *kasi-ra*. However, *ra* has a VOICE feature as its bottom feature, while the feature that we are currently trying to lexicalise is ASP (see Table 2, (e)). It thus seems that the derivation has “gone too far” to be able to lexicalise VOICE. If we look again at Table 2 (d) we can see that *ye* has ASP as a bottom feature. The reason why we cannot insert it at (42) is because the bottom of *ye* is binary, i.e. it has an empty feature and ASP. This means that *ye* can only be lexicalised if a new workspace is opened. Opening a new workspace is step (40e) in the algorithm. Before the syntax gets to that step though, it will first try step (40c). However, that step, which involves movement of a complex left branch (an independent workspace), is at this point irrelevant, since there are no complex left branches yet to be moved. So syntax skips to the next step, which turns out to be backtracking (40d). This step in the algorithm forces the syntax, after repeated attempts at unsuccessful lexicalisation, to undo the derivation and try the next option for the previous cycle. This is exactly the type of procedure that we need to repair the derivation that has gone “too far”. So for the derivation under discussion this means that the syntax undoes the merger of ASP, and goes back to the previous cycle, i.e. to VOICE, i.e. back to (37). Instead of lexicalising VOICE without any evacuation with the verbal predicate *kasi*, which was the first step in (40), the next option is to do an evacuation movement, (40b). That leads to the structure in (43).

(43)

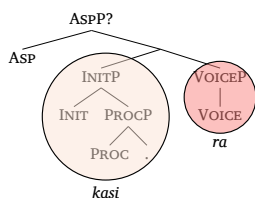


(44)

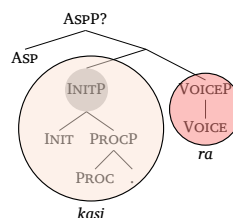


The right branch of (43) can now be lexicalised by *-ra*, the allomorph for the intransitive PFV (Table 2(e)). There is thus a successful lexicalisation (44). In a next step, ASP is again added to the derivation (45). Since there is no matching lexical item for this structure, an evacuation movement is performed. The closest node that contains the tree foot is marked by grey shading in (46) and it is this one, i.e. the specifier INITP, that gets moved, leading to the structure in (47). The right branch can be lexicalised by *ra* (Table 2(e)), leading to a successful lexicalisation, depicted in (48).

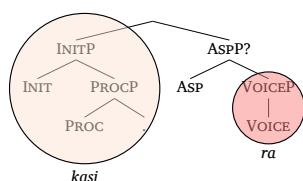
(45)



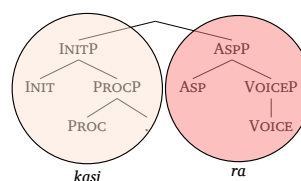
(46)



(47)

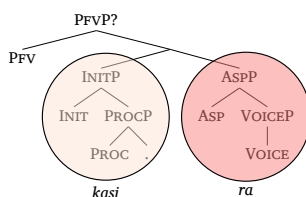


(48)

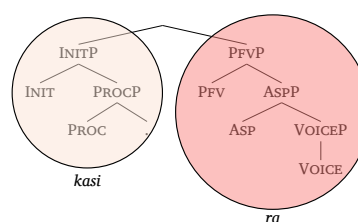


Next the feature relevant for the perfective, which we refer to as PFV, is merged (49). Again, the syntax will have to do an evacuation of the specifier in order to be able to lexicalise PFV together with *ra*, as in (50).

(49)

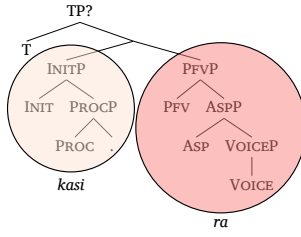


(50)

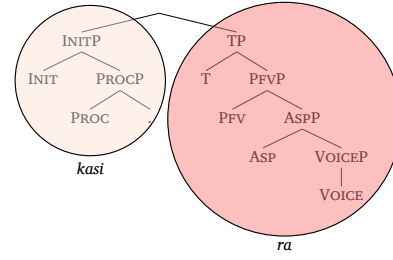


The same goes for the next feature in line, i.e. T (51), which will also trigger evacuation movement before it leads to the successful lexicalisation in (52).

(51)



(52)

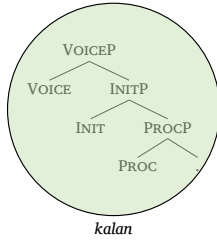


We have now come to the end of the derivation of the INTRANS.PFV, i.e. of *kasi-ra*. Due to the algorithm (40), which ranks backtracking (40d) before the formation of complex left branches (40e), i.e. independent workspaces with a binary foot, it was not an option for transitive *ye* to kick in, and we derived the grammatical *kasi-ra*. We discuss in the next section 4.1.2 how for transitive structures *ra* will never be a candidate, and the formation of a new workspace, a complex left branch, will be the only option.

4.1.2 The derivation of TRANS.PFV *ye kalan*

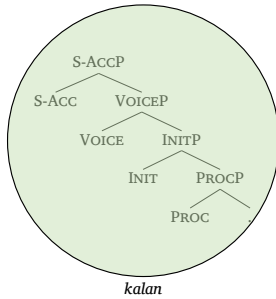
To derive a transitive predicate in the perfective, we start with a transitive predicate, in this case *kalan* ‘read’, which is stored in the lexicon as in Table 2(a). Everything will be the same for the derivation of *kalan*, until the point where VOICEP is lexicalised, as in (53).

(53)



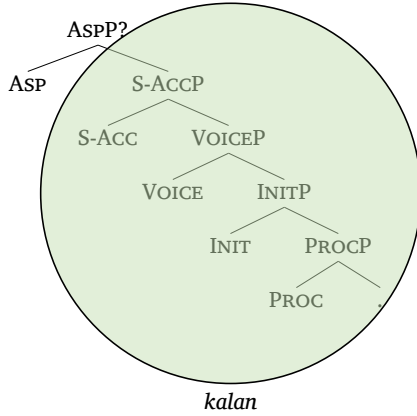
However, different from intransitive predicates, the derivation of a transitive verb comes with the additional instruction to combine with an argument that will need accusative case, i.e. the feature S-ACC (Caha 2009), which stands for “select an accusatively marked argument”, will be merged, and can be lexicalized together with any transitive predicate, as in (54).

(54)



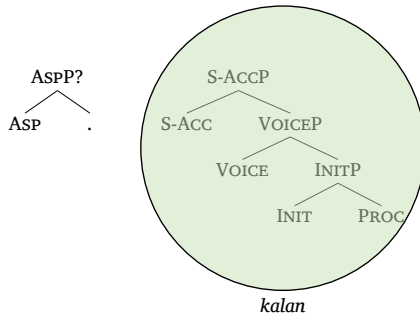
Next ASP will again be merged, and just like before, there will be no lexical item in Table 2 to lexicalise the structure in (55).

(55)



Evacuation will not lead to a lexicalisation, move will be irrelevant since there is nothing to move, and backtracking will not work either.⁸ This brings us to the next step in the algorithm, i.e. complex left branch (CLB) formation (40e), or the formation of a new workspace. This boils down to merging a new workspace with a binary foot, which consists of the empty feature and F, i.e. the new feature required by the FSEQ. This F will then project in the main sequence as a complex left branch, and precede (PRE) the constituent it combines with. At this particular point in the derivation the empty feature will combine with ASP, leading to ASPP (56). Just like before, this phrase will need to get lexicalised, and hence syntax interfaces with the lexicon.

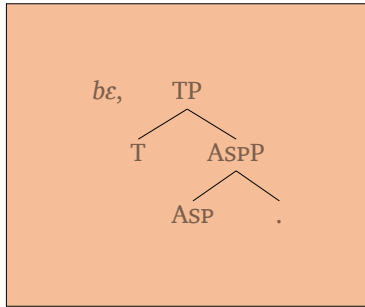
(56)



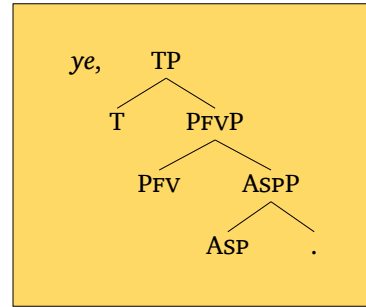
This time there are two different lexical items in a tie to lexicalise ASPP thanks to the Superset Effect (23), i.e. ASPP is a subconstituent of two lexical items *ye* (Table 2(d), repeated here as (57)), and *bε* (Table 2(c), repeated as (58)). Since *bε* contains fewer features, it will at this point be the best candidate for lexicalisation. This is due to the Elsewhere Principle (59) (Kiparsky 1973), which regulates the competition between different candidates for lexicalisation.

⁸Backtracking will not work, because when ASP is un-merged, a new step will be tried at the cycle of S-ACC, i.e. evacuation. However, evacuation of S-ACC will not lead to a lexicalisation, since there is no lexical item in the lexicon (Table 2), that has S-ACC as its foot. Even if syntax tries to backtrack further, into VOICE for instance, the derivation will run into problems. VOICE will be able to get lexicalised by *-ra*, just like with intransitive predicates, but as soon as S-ACC will be merged no lexicalisation will be possible, requiring to undo the evacuation step of the Voice-cycle. Cutting things short, whatever syntax will try, backtracking will not lead to a solution to lexicalise ASP.

(57)



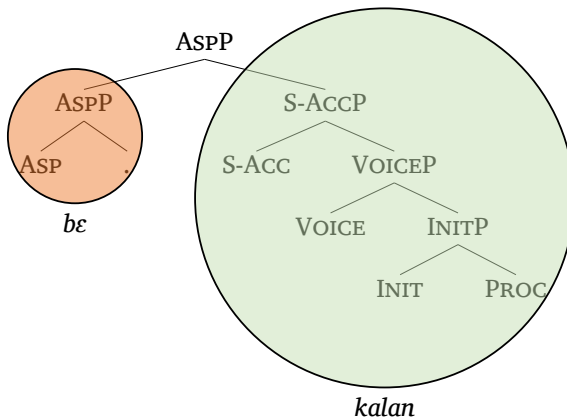
(58)

(59) *The Elsewhere Principle*

In case two rules, R_1 and R_2 , can apply in an environment E , R_1 takes precedence over R_2 if it applies in a proper subset of environments compared to R_2 .

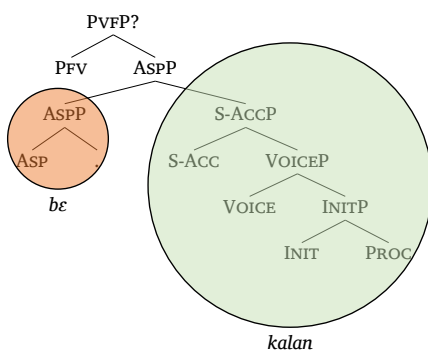
The two workspaces will then be merged, and ASP will project in the main spine, as in (60).

(60)



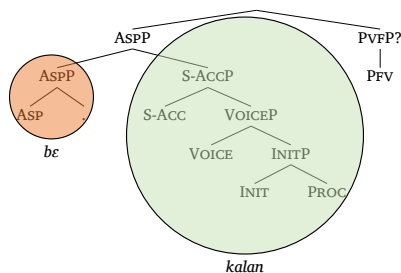
Since we wanted to derive a perfective, the next feature in line will be PFV. PFV will not be lexicalisable with the structure in (61), i.e. there are no matching lexical items for this structure in the lexicon.

(61)



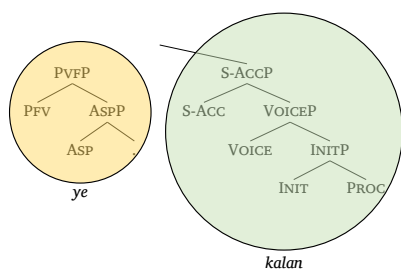
Even the second step of the algorithm, i.e. evacuation, will not lead to any outcome, as in (62), since there are no lexical items in the lexicon that have a foot in PFV.

(62)



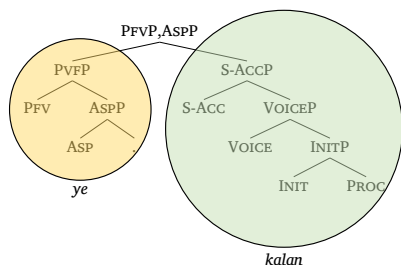
What we actually need is for PVFP to be merged inside the CLB that has just been created. This situation is depicted in (63). This leads to a successful lexicalisation: *ye* can be inserted, as in (63).

(63)



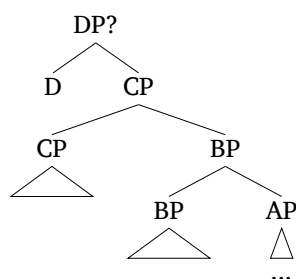
Now the two features of the complex left branch can project in the main spine, as in (64). This step of merging a feature with a lower workspace can be subsumed under the step move in the algorithm, i.e. (40c) (Starke 2024). The way step (40c) in the algorithm works needs some more discussion, in particular the reason to consider this as an instance of move.

(64)



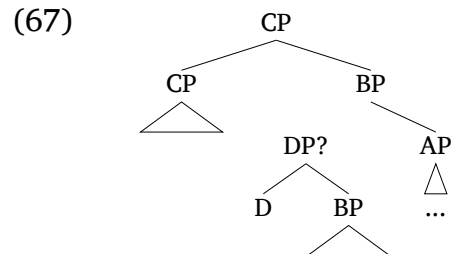
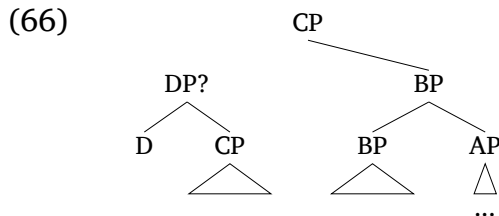
Consider a situation in which more than one CLB or separate workspace is present, as abstractly depicted in (65), where the current cycle has merged D in the main spine, and there are two CLBs resulting from the earlier derivation, BP and CP.⁹

(65)

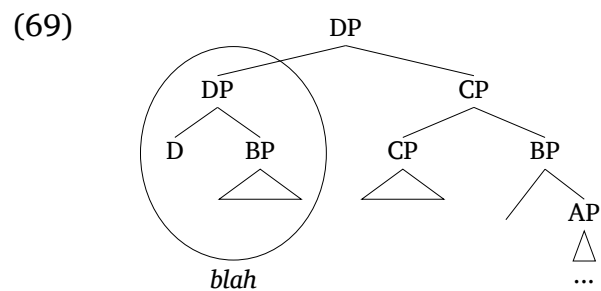
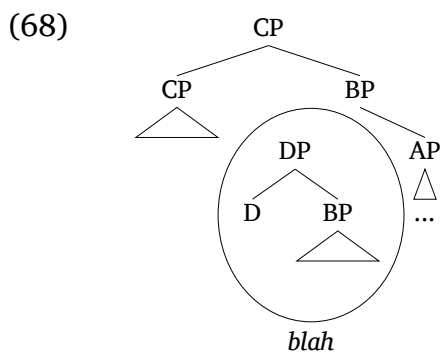


⁹Michal Starke discussed the Merge of Fs with lower CLBs and their subsequent move in a series of seminars in 2024 and 2025 at Masaryk University, Université Paris Cité and KU Leuven (Brussels, CRISSP). All errors in representing his ideas are of course mine.

Assuming all the derivational steps in the algorithm before (40c) to have failed, the move option involves trying to find a workspace other than the current one that can provide a lexicalisation for D. If the newly merged feature D cannot be merged with the main spine, it will first be merged with the closest lower workspace CP, as shown in (66). If that attempt is unsuccessful, it will be merged with BP (67) (and successively further lower down if there were more sub-workspaces).



Now suppose that there is a lexical item *blah* in the lexicon that allows D to be lexicalised with BP. Lexicalisation will then be successful in the BP workspace (68). However, DP is still floating now, and the feature D needs to project in the main workspace, at the root of the tree. This will lead to the move of the CLB DP to the original merge position of D, where it will also project its label, as shown in (69).

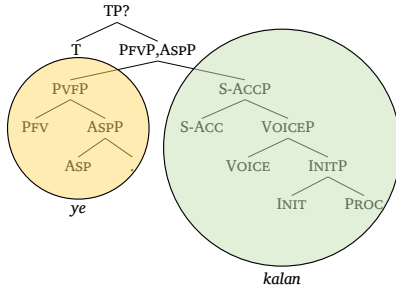


One can easily see how this abstract case constitutes an instance of movement.¹⁰ In the Bambara case, the movement could be said to be vacuous, in that it does not lead to a linear reordering of phrases, but the essence of the procedure is the same. That is, what happens in (64) is that the new feature PVF is merged in the closest CLB (ASPP). In (64), this is successful, and the enriched CLB projects into the main workspace.

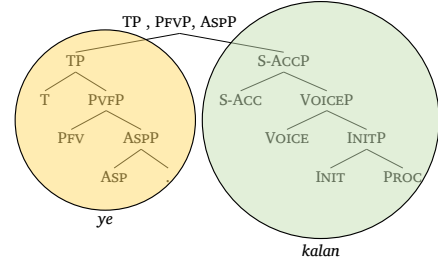
If we now go back to our derivation in (64), then in the next step T is added to the derivation, as in (70). Just like before, evacuation movement will be tried, without success. Next, the move step (40c) will be tried, and T will be added to the CLB and project, in the same way as we described for the PFV feature just before, leading to a successful lexicalisation and projection of TP in the main spine, as in (71).

¹⁰The move step in the algorithm allows one to capture the effects of head-movement. To make the abstract case discussed in (68)-(69) somewhat more concrete, one can take DP/BP to be the auxiliary *are*, CP the negator *not*, and AP the verb *singing*, as in *They are not singing*, where the auxiliary has moved across the negation (Pollock 1989). See De Clercq (2020a) for more discussion of the question how head-movement can be captured in Nanosyntax as an instance of feature-driven movement, another implementation of the move step in (40c).

(70)



(71)



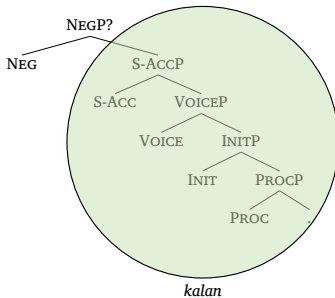
We have now shown how for transitive predicates the backtracking option that we discussed for *-ra* will never yield a successful lexicalisation. This is due to the fact that for transitive predicates a feature S-ACC will be merged in syntax, which prepares the predicate for the selection of an accusatively marked argument. Since *-ra* does not contain this feature, it will never be a successful candidate for the lexicalisation of transitive structures. We showed how this will allow the algorithm to go to the next step, i.e. a complex left branch formation, allowing for *ye* to kick in and to lexicalize all TAM-features, including PFV. In the next section, we discuss how the allomorphy and the split that we discussed for the positive gets neutralised under negation.

4.2 Deriving neutralisation in NEG.PFV

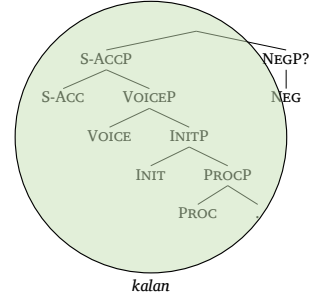
We want to derive *ma kalan* and *ma kasi* in this section, the negative form of the PFV for transitive and intransitive verbs respectively. In (19) we assumed that the position for sentential negation is above S-ACC, the feature responsible for instructing the predicate to select an accusatively marked argument. In what follows we will first show how we can derive the negative neutralisation in the PFV. Next, we will show this how any higher position for negation would allow for the derivation of ungrammatical structures.

We begin with the derivation of the transitive negative perfective predicate *ma kalan*. Up until the derivation of S-ACC everything is similar to what we discussed for the positive PFV transitive predicate in section 4.1.2, so we will start from the structure in (54). Next NEG will be merged in syntax, as in (72). The regular evacuation movement will be tried, (73), but this will not lead to any lexicalisation, because there is no lexical item in Table 2 that has a unary foot in NEG.

(72)



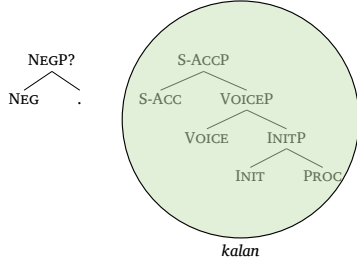
(73)



Just like we explained for the positive derivation, the next step in the algorithm, i.e. (40c) in (40), can be skipped because there is no CLB yet, so we cannot add F to an existing workspace and move it. Also backtracking will not lead to anything. Even if we backtrack into VOICE, and do the next step for that cycle, i.e. evacuation, the subsequent *-ra* will have to be undone as soon as S-ACC kicks in, because *-ra* will not be able to lexicalise VOICE

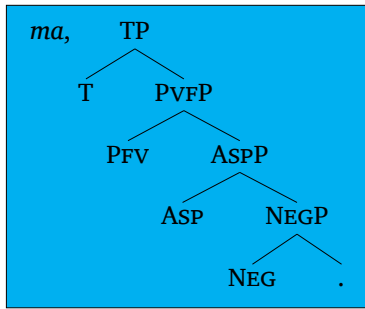
and S-ACC together, and will definitely not help in lexicalising the required NEG feature. Consequently, only merge of a new workspace, i.e. (40e), can rescue the derivation here. A new workspace is opened, and the empty feature is combined with NEG, upon which the syntax interfaces with the lexicon, (74).

(74)

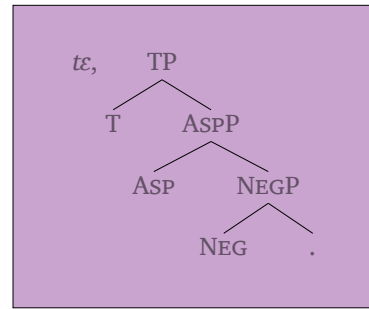


There are two lexical items in the Bambara lexicon that we discuss in this paper that have a foot in NEG. We repeat them here in (75) and (76).

(75)

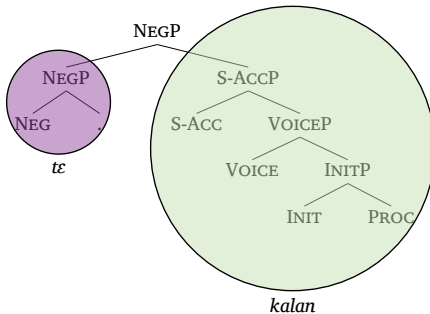


(76)



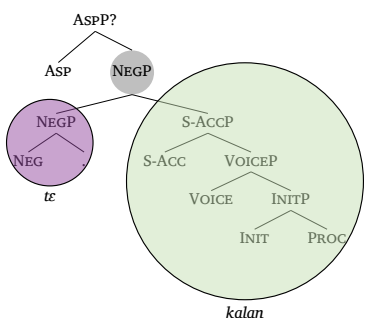
At (74) the best candidate for lexicalisation, in line with the Elsewhere Principle, (59), is (76), leading to (77).

(77)

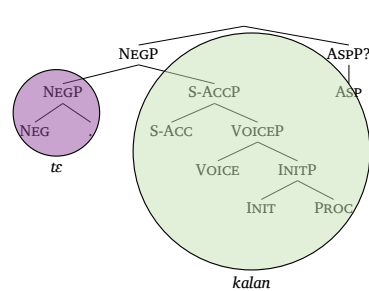


The next feature in line is default ASP (78), which after evacuation will also not lead to a successful lexicalisation (79) given the available lexical items in Table 2.

(78)

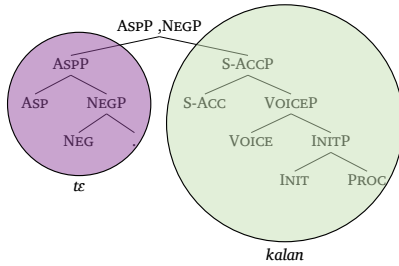


(79)

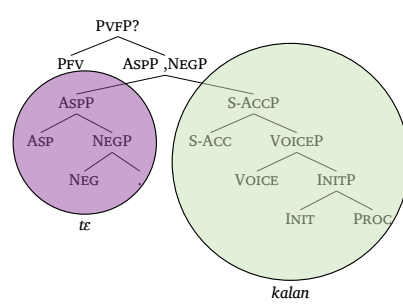


However, at this point, there is an additional workspace available and so the newly merged F, i.e. ASP, can be added to the lower CLB, as in (80). This is the move-step from the algorithm in (40), which we discussed in section 4.1.2. (76) will still be the best candidate for lexicalisation. Both features project in the main spine. Next PFV is merged (81), and the syntax will have to go through the same steps to eventually add the feature to the CLB, leading to the lexicalisation of the negative PFV marker *ma* (82), which now is the best match for the CLB in (81), overriding the previous lexicalisation with *ma*. After merge with the main spine, PFV in the CLB will project in the main spine (82).

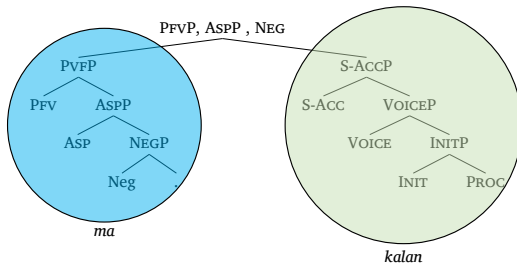
(80)



(81)

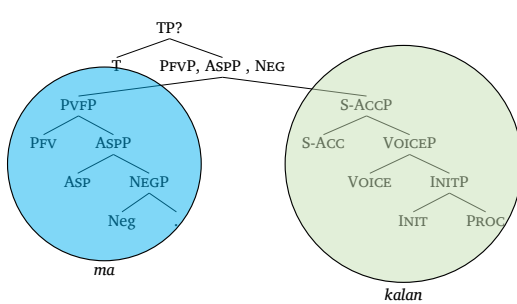


(82)

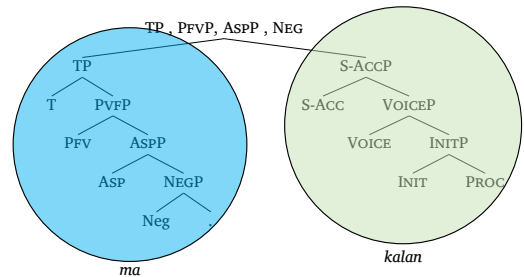


The last feature relevant for TAM and the verbal predicate is T. The same steps as before will be walked through (evacuation, and then move) leading to (84).

(83)

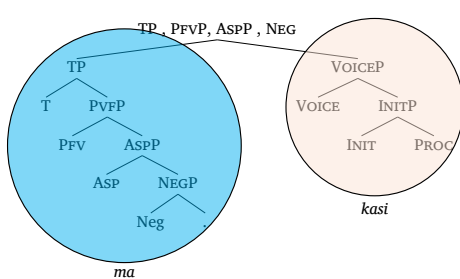


(84)

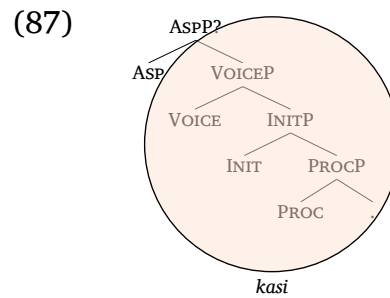
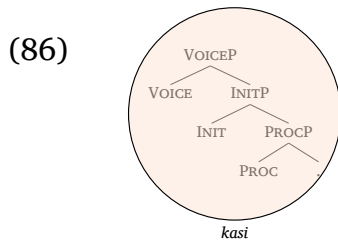


With intransitive predicates, the steps are exactly the same. (85) illustrates the output of the derivation of *kasi* 'cry' in the NEG.PFV *ma kasi*.

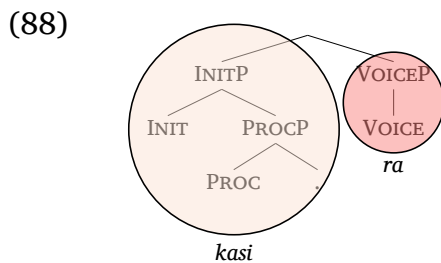
(85)



The final derivation is immediately also the reason why NEG sits where it sits in (19), i.e. just above S-ACC. Let us have a closer look at why we could not put it higher. Imagine NEG was just above PFVP in the FSEQ. In that case, if we were to derive a PFV with the intransitive predicate *kasi*, we would derive an ungrammatical structure. Let me explain why. So imagine we derive the predicate *kasi* just until Voice, as we did before in section 4.1.1 for the positive derivation, i.e. the structure would be as in (86), repeated here from (38).



The next feature in line for an intransitive predicate would be ASP, since there is no S-ACC in an intransitive structure, (87) repeated from (39). Just like before, there will be no way to lexicalise ASP, not by immediate lexicalisation nor by evacuation movement or move. We refer the reader to section 4.1.1 for the different unsuccessful steps in the lexicalisation, because they are exactly the same. Hence, the merge of ASP will trigger backtracking into VOICE, for which the next step in the algorithm, i.e. evacuation, will be tried, resulting in the lexicalisation of *kasi-ra* as illustrated in (88) repeated from (44).



Next ASP, the feature that gave rise to the backtracking will be merged again and - after evacuation movement - will get lexicalised with *-ra*, just like before. In the next merge step it would be time for NEG to be merged, but at the point when this happens, there is no need to undo *-ra* anymore, since this needed to kick in - via backtracking - to lexicalise ASP. After going through various steps of the algorithm, a new workspace will need to be formed to be able to lexicalise NEG, just like we discussed before, eventually leading to the lexicalisation of the ungrammatical **ma + kasi-ra*. Putting NEG in any other position higher than S-ACC will lead to ungrammatical structures in the derivation of intransitive predicates. As such, in spite of the fact that there are no other clear ways to see where negation sits in the syntax of Bambara, due to the highly fusional nature of its TAMP-markers, Nanosyntax and the algorithm offer a way to diagnose the position of negation.

We have shown in this section how negative neutralisation follows from the combination of the structure of the FSEQ with a low position for sentential negation, the internal structure of lexical items, and the Nanosyntactic algorithm. All of these elements are an intrinsic part of the Nanosyntactic toolbox, and hence no additional mechanisms are needed to capture

neutralisation or the allomorphy discussed before. Before we conclude we want to briefly situate the derivations that we just looked at within the syntax of a full clause, i.e. we want to say something - tentatively - about where the arguments sit within this framework.

5 A final note: the arguments

One of the questions the reader may be left with at this point is how this syntax for the verbal domain interacts with the insertion of the arguments. Or put differently: where exactly in this structure building process do the arguments come in? In what follows I do a tentative and programmatic suggestion that may contribute to the discussion on that topic (see also proposals by Starke (2014), Savu (in preparation)), but it goes without saying that a worked out proposal will have to be postponed to future research.¹¹

In traditional generative syntax the introduction of the arguments in the structure does not come with a dedicated projection of a feature in the main spine. The internal argument is usually introduced as the complement of the verbal head, and the external argument as the specifier of the VP (in Government and Binding) or of a vP (Chomsky 1995), or of a VoiceP (Kratzer 1996).¹² In addition, different functions are usually clustered within one head; for instance verbalisation is often clustered with the introduction of an external argument. Even approaches to Voice, which separate out different heads for verbalisation, Voice and applicative arguments (Pylkkänen 2002, Harley 2013, Wood 2015) also introduce the external argument in the specifier of the Voice-projection, ultimately often clustering different functions (causation and agentivity) in one feature/head.

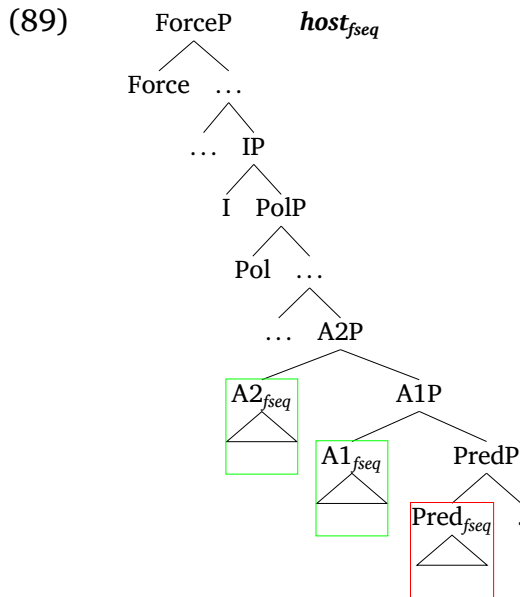
Since specifiers in Nanosyntax project a feature, just like the head of a phrase projects a feature (Starke 2004), we cannot say that the head VOICE, for instance, is responsible both for VOICE morphology on the predicate, and for the introduction of an external argument. At least two different features would be required from a Nanosyntactic perspective.

What I want to propose is that the features from Ramchand's work (Ramchand 2008), i.e. RES, PROC and INIT (see also (19)) allow for the verb to denote a resulting, dynamic, or causing event, but these features themselves do not introduce the arguments, unlike in Ramchand's work (and much other related work), where the arguments constitute the specifiers of these heads. These features merely prepare the predicate to combine with particular types of participants: agents, patients, experiencers, goals, etc. Which participants a predicate can combine with is, I assume, on the one hand determined by the syntactico-semantic features RES, PROC and INIT the verb lexicalizes, but also by the conceptual information that comes with the predicate. Next, the predicate combines with features that will instruct the syntax to combine, in a later stage of the derivation, with one or two or three of these potential participants. We have used two of these instructional features in the course of this paper: VOICE, and S-ACC, but - needless to say - there are more layers within this domain for argument realisation. For instance a layer PASS (Passive) could instruct to suppress a

¹¹Starke (2014) suggests an account for argument selection/introduction in terms of peeling and stored verbal predicates that consist of peeled off NPs in their structure. Along the same lines Savu (in preparation) elaborates on this, and applies this to Hindi/Urdu split ergative data. Savu (in preparation)'s proposal involves complex lexical entries for verbal predicates with pointers to guarantee flexibility regarding the optionality of arguments.

¹²In some theories Voice only plays a role in how arguments are realised in A-positions, while the Merge position for the external argument is vP Collins (2005).

participant, or a layer like APPL will allow the predicate to combine with an additional applicative argument. If only VOICE is merged, the predicate will only be allowed to combine with one of its participants or its sole participant. If S-ACC is also merged on top of VOICE, this will allow the predicate in a later stage to combine with a second, accusatively marked, participant. Once the features related to argument realisation are in place, see also (19), all the TAM features will be merged. When the predicate is complete it will project in the main spine as a CLB, as depicted in (89), where the red square marks the CLB of the verbal predicate.



The derivation depicted in (89) shows a derivation that has at its bottom an empty feature (marked by the dot), which merges with the CLB of the predicate. This CLB has grown in accordance with the same algorithmic principles we discussed earlier in the paper, i.e. step by step. After the predicate is in place, the relevant arguments for that particular derivation will be merged. The instructions for that are contained in the predicate CLB. If a VOICE and S-ACC feature are present in the predicate structure, two participants will be merged as arguments. They will each project in the main sequence thanks to a slot that gets opened for them due to the instructions in the predicate. I will call the projecting features - for now - ARGUMENT1 (A1) and ARGUMENT2 (A2), respectively (marked by the green squares). Next the host spine will grow and will merge projections like IP, PolP, and CP, attracting, via feature-driven movement, complex left branches, or subconstituents of them, from lower down.

This type of proposal leaves the account developed in this paper intact. The only thing the reader needs to be aware of is that the account of Bambare in the previous sections presents the syntax for the predicate FSEQ (in the red box) only. It is only in later stages of the derivation that this FSEQ combines with other FSEQs to give rise to a full clausal structure. While there are many questions left open here, I hope to have clarified, albeit in a programmatic way, how the insertion of arguments could work in Nanosyntax, and that the domain of discussion within this paper is confined to what is labeled as the Pred_{fseq} in (89). I leave the development of a full theory of argument selection to further research.

6 Conclusion

In this paper I discussed a recalcitrant set of data from Bambara, which displays an allomorphy pattern in the perfective triggered by the argument structural properties of the verb. We showed how the allomorphy can be captured in terms of the structural size of the TAM-markers: one of the two TAM-markers, *ra*, has a VOICE feature as its anchor, but not a feature S-ACC, blocking its use with transitive predicates. In addition, we discussed how the neutralisation of this split can be understood as a consequence of the fact that the negative perfective marker *ma* is not specified for any Voice- or S-Acc features, allowing these to be lexicalised by the predicate, and leading to the neutralisation of the perfectivity split. In addition, the paper discussed how the Nanosyntactic algorithm can help to diagnose the position of negation in a highly fusional language like Bambara.

References

- Baunaz, Lena & Eric Lander. 2018. Deconstructing categories syncretic with the nominal complementizer. *Glossa: a journal of general linguistics* 3(1). 31.1–27.
- Burzio, Luigi. 1986. *Italian syntax*. Dordrecht: Reidel.
- Caha, Pavel. 2009. *The Nanosyntax of Case*. Tromsø: University of Tromsø dissertation.
- Caha, Pavel. 2022. Minimalism and nanosyntax: Reconciling late insertion and the Borer-Chomsky conjecture. In Kleanthes K. Grohmann & Evelina Leivada (eds.), *The Cambridge handbook of minimalism*. Cambridge University Press.
- Caha, Pavel, Karen De Clercq & Guido Vanden Wyngaerd. 2019. The fine structure of the comparative. *Studia Linguistica* 73(3). 470–521. doi:<https://doi.org/10.1111/stul.12107>.
- Chomsky, Noam. 1981. *Lectures on government and binding*. Dordrecht: Foris.
- Chomsky, Noam. 1995. *The minimalist program*. Cambridge, Massachusetts: MIT Press.
- Collins, Chris. 2005. A smuggling approach to the passive in English. *Syntax* 8(2). 81–120.
- Creissels, Denis. 1988. *Le système tonal du bambara standard*. Grenoble: Centre de linguistique africaine de l'université de Grenoble.
- Creissels, Denis. 1992. Tonologie du bambara: bilan et perspectives. *Mandenkan* 24. 1–45.
- Creissels, Denis. 2007. A sketch of Bambara argument structure. Workshop on Grammar and Processing of verbal arguments.
- De Clercq, Karen. 2020a. *The morphosyntax of negative markers. A nanosyntactic account*. Mouton de Gruyter.
- De Clercq, Karen. 2020b. Tense and sentential negation: a typological perspective. In Elena Tribushinina & Mark Dingemanse (eds.), *Linguistics in the Netherlands*, vol. 37. 71–89. Benjamins.

- De Clercq, Karen, Pavel Caha, Michal Starke & G. Vanden Wyngaerd. to appear. Nanosyntax. In Hilary Nesi & Petar Mitin (eds.), *Encyclopedia of language and linguistics*. Elsevier.
- De Clercq, Karen, Pavel Caha, Michal Starke & Guido Vanden Wyngaerd. 2025. Nanosyntax: state of the art and recent developments. In Pavel Caha, Karen De Clercq & Guido Vanden Wyngaerd (eds.), *Nanosyntax and the lexicalisation algorithm*. 1–44. Oxford: Oxford University Press.
- Dumestre, Gérard. 2003. *Grammaire fondamentale du bambara*. Paris: Karthala.
- Ermisch, Sonja. 2013. The structure of Bambara. Handout, Goethe Universität Frankfurt.
- Giorgi, Alessandra & Fabio Pianesi. 1997. *Tense and aspect*. Oxford: Oxford University Press.
- Harley, Heidi. 2013. External arguments and the Mirror Principle: On the distinctness of Voice and v. *Lingua* 125. 34–57.
- Hewson, John. 2016. Bambara. In Derek Nurse, Sarah Rose & John Hewson (eds.), *Tense and aspect in Niger-Congo*. Royal Museum for Central Africa.
- Idiâtov, Dmitrij. 2012. Clause-final negative markers in southeastern bamana dialects: a contact-induced evolution. *Africana Linguistica* 18. 161–191.
- Kiparsky, Paul. 1973. ‘Elsewhere’ in phonology. In Stephen Anderson & Paul Kiparsky (eds.), *A festschrift for Morris Halle*. 93–106. New York: Holt, Rinehart & Winston.
- Klein, Wolfgang. 1992. The present perfect puzzle. *Language* 68(3). 525–552.
- Koopman, Hilda. 1992. On the absence of case chains in Bambara. *Natural Language & Linguistic Theory* 10. 555–594.
- Kratzer, Angelika. 1996. Severing the external argument from its verb. In Johan Rooryck & Laurie Zaring (eds.), *Phrase structure and the lexicon*. 109–137. Dordrecht: Kluwer.
- Pollock, Jean-Yves. 1989. Verb movement, universal grammar and the structure of IP. *Linguistic Inquiry* 20. 365–424.
- Pylkkänen, Liina. 2002. *Introducing arguments*: Massachusetts Institute of Technology dissertation.
- Ramchand, Gillian. 2008. *Verb meaning and the lexicon*. Cambridge: Cambridge University Press.
- Savu, Carmen. in preparation. *Argument structure and case alignment in Nanosyntax. a case study on Hindi/Urdu split ergativity*. Norway: UiT The Arctic University of Norway dissertation.
- Starke, Michal. 2004. On the inexistence of specifiers and the nature of heads. In Adriana Belletti (ed.), *Structures and beyond*, vol. 3 The Cartography of Syntactic Structures. 252–268. Oxford: Oxford University Press.

- Starke, Michal. 2009. Nanosyntax: A short primer to a new approach to language. *Nordlyd* 36(1). 1–6.
- Starke, Michal. 2014. Why you can't stay away from clitics. Talk presented at GIST 7, Ghent.
- Starke, Michal. 2018. Complex left branches, spellout, and prefixes. In Lena Baunaz, Karen De Clercq, Liliane Haegeman & Eric Lander (eds.), *Exploring nanosyntax*. 239–249. Oxford: Oxford University Press.
- Starke, Michal. 2022. Nanoseminar. Lectures at Masaryk University, Brno.
- Starke, Michal. 2024. Nanosyntax seminar. Lectures at Masaryk University, Fall 2024.
- Wood, Jim. 2015. *Icelandic morphosyntax and argument structure*. New York, Dordrecht, London: Springer.